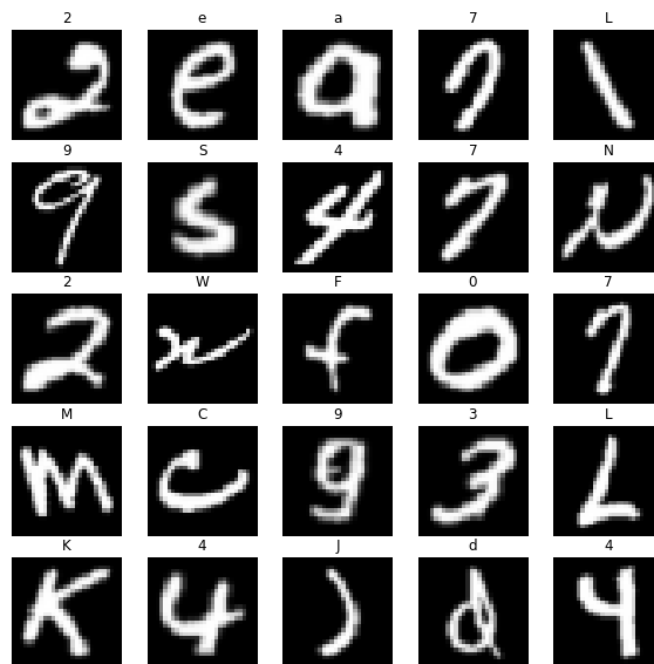




LEPL1507 - Projet IV (Mathématiques Appliquées)

Rapport final

Opération Chiffre Fantôme

La Course aux Classifieurs Invulnérables

Groupe 02

Adrien Anciaux
Yanis Bouchema
Léopold Dallemagne

Donny Fotsing Talla
Gaspard Martinussen
Ganza Ntigura

Professeurs :
Gautier Krings
Estelle Massart

Assistants :
Roy Makhoulouf
Bastien Massion

17 avril 2026

Table des matières

1	Introduction	1
2	Phase 1 : Conception du classifieur de base	1
2.1	Approche MLP	1
2.1.1	Introduction	1
2.1.2	Fonction de coût	1
2.1.3	Rétro-propagation	2
2.1.4	Transition vers l'architecture convolutive (CNN)	3
2.2	Approche CNN	3
2.2.1	Introduction	3
2.3	Techniques de test pour les hyperparamètres	3
2.3.1	Architecture des couches	4
2.3.2	Le taux d'apprentissage	6
2.3.3	Le scheduler de ce taux d'apprentissage	6
2.3.4	Batch size et nombre d'epochs	8
2.4	Performances initiales	9
3	Phase 2 : Stratégies d'attaque adversariale	9
3.1	Contraintes et méthodologie	9
3.2	Approche d'attaque n°1 : PGD+	10
3.2.1	Améliorations apportées à PGD	10
3.2.1.1	Momentum.	10
3.2.1.2	Diversification des fonctions de perte.	10
3.2.1.3	Pas adaptatif.	11
3.2.1.4	Multi-restart.	11
3.2.2	Résultats	11
3.2.3	Analyse de cas concrets	12
3.2.3.1	Cas 1 : échec sur 'Y'.	12
3.2.3.2	Cas 2 : succès sur '1'.	12
3.3	Approche d'attaque n°2	13
3.3.1	Analyse des performances	14
3.4	Comparaison entre nos deux attaques	15
3.4.1	Formulations mathématiques comparées	15
3.4.2	Mécanismes :comment chaque méthode construit δ	15
3.4.3	Observation clé	16
4	Phase 3 : Le classifieur robuste	16
4.1	Techniques de défense et stratégies de robustesse	16
4.1.1	Entraînement adversarial de base	16
4.1.2	Transition vers l'algorithme TRADES et la divergence KL	17
4.1.3	Régularisation par Lissage des Étiquettes (Label Smoothing)	17
4.1.4	Progression du Budget de Perturbation (Scheduling)	18
4.1.5	Évaluation croisée et optimisation des hyperparamètres	18
4.1.6	Résultats finaux de robustesse	19
5	Conclusion	19
6	Utilisation de l'IA	19

1 Introduction

Dans le cadre de notre cours de Projet IV en Mathématiques Appliquées, nous devons développer un classifieur le plus robuste possible afin de correctement labelliser des chiffres et des lettres. L'enjeu est de taille : des fraudeurs exploitent des failles dans le système de lecture automatique de chèques de la BES (Banque Européenne de Sécurité), en menant des attaques adversariales discrètes sur les documents. Pour faire face à ce fléau, le CISO (Chief Information Security Officer) de la BES nous a chargé, dans le cadre du Hackathon Cybersécurité IA 2026, de renforcer ce système pour lui permettre de déjouer ces attaques, et d'élaborer de telles attaques, afin de pouvoir mieux y faire face.

2 Phase 1 : Conception du classifieur de base

2.1 Approche MLP

2.1.1 Introduction

Comme premier modèle, nous avons d'abord essayé d'entraîner un modèle de Multilayer Perceptron (MLP). Notre première source a été la mini-série de 3blue1brown sur les réseaux neuronaux [1], qui en explique les fondamentaux, et implémente un MLP. Nous avons donc commencé par cette approche pour notre classifieur.

Un MLP est une architecture de réseau de neurones artificiels feedforward, composée de plusieurs couches entièrement connectées. Cela signifie que chaque couche applique une transformation linéaire à ses entrées (qui sont les neurones de la couche précédente), suivie d'une fonction d'activation potentiellement non-linéaire. La couche 1 correspond donc à l'équation suivante :

$$a^{(l)} = \sigma(W^{(l)}a^{(l-1)} + b^{(l)})$$

La fonction sigma peut correspondre à plusieurs fonctions, par exemple, une sigmoïde, ce qui permet de garder la valeur de sortie entre 0 et 1, ou ReLU, qui garde uniquement les valeurs positives. Donc chaque neurone dépend de tous les neurones de la couche précédente, mais pas de neurones issus de la même couche ou une couche ultérieure, c'est pour cela que c'est feedforward. Un biais, b , est également ajouté pour chaque neurone.

Dans notre cas, nous devons donc transformer notre image de 28x28 pixels en un vecteur de 784 éléments, et fournir en sortie une couche de 47 neurones. Par contre, le nombre et la taille des couches intermédiaires, celles qui sont donc "invisibles", sont des hyperparamètres que nous devons fixer. Après plusieurs tentatives, nous avons remarqué que le plus efficace était d'avoir deux couches intermédiaires, chacune de taille 512.

2.1.2 Fonction de coût

Pour entraîner le modèle, il faut d'abord déterminer la fonction de coût que l'on veut minimiser. Pour cela, nous avons choisi d'utiliser la CrossEntropyLoss, qui applique successivement LogSoftMax et NLLoss. Pour la comprendre, définissons d'abord softmax, qui transforme des logits en une distribution de probabilités :

$$\text{softmax}(x)_i = \frac{e^{x_i}}{\sum_{k=1}^K e^{x_k}}$$

Si on en prend le logarithme, on obtient, puisque $\log(e^{x_i}) = x_i$:

$$\log(\text{softmax}(x)_y) = x_y - \log\left(\sum_{k=1}^K e^{x_k}\right)$$

Ensuite, on applique NLLoss, ce qui revient, comme on a déjà des log-probabilités, à prendre l'opposé de la log-probabilité de la classe correcte.

$$\text{NLLoss}(\log(\text{softmax}(x)_y)) = -\log(\text{softmax}(x)_y)$$

Cette opération est logique : on cherche à maximiser la probabilité de la bonne classe, donc minimiser son opposé. Contrairement aux moindres carrés, la cross-entropy pénalise une faible probabilité attribuée

à la classe correcte, et ne regarde pas les autres classes. Avec les moindres carrés, il y a une recherche d'équité quand à la valeur (la plus faible possible) des mauvaises classes. Dans le cas d'un classifieur, il est peu intéressant d'essayer d'harmoniser les mauvaises catégories.

En combinant les deux expressions précédentes, on obtient la définition de notre fonction de coût [2] :

$$\text{CrossEntropyLoss}(x, y) = -x_y + \log \left(\sum_{k=1}^K e^{x_k} \right)$$

Nous avons donc maintenant une fonction coût que nous voulons minimiser. Pour ce faire, nous allons calculer le gradient de cette fonction de coût par rapport à tous les paramètres (poids et biais) du réseau. Conceptuellement, comme l'illustre de manière très visuelle la série de 3blue1brown [1], on peut imaginer la fonction de coût C comme une vaste surface topographique dans un espace à très haute dimension. Notre réseau, avec son initialisation de paramètres actuelle, se trouve à des coordonnées spécifiques sur cette surface. L'objectif de la phase d'apprentissage est de naviguer sur cette surface pour atteindre le point le plus bas, représentant l'erreur minimale.

Le concept mathématique central pour s'orienter sur cette surface est le gradient, noté ∇C . Il indique la direction de la pente la plus raide vers le haut. Pour minimiser l'erreur et aller vers le minimum, on doit donc emprunter la direction opposée, $-\nabla C$. L'équation de mise à jour de la matrice des poids W s'écrit alors :

$$W_{k+1} = W_k - \eta \nabla C(W_k)$$

où le scalaire η est le taux d'apprentissage (learning rate), définissant l'amplitude du pas effectué à chaque itération (nous verrons plus tard quelles stratégies de choix de ce learning rate nous avons adopté). Mais nous ne calculons pas, avant de modifier nos poids, le gradient sur toutes les données d'entraînement. C'est en réalité sur un lot aléatoire de celles-ci d'une certaine taille B que nous approximations le gradient $\hat{g}_B$ pour modifier les poids à chaque itération. Cet échantillonnage introduit un bruit statistique ϵ :

$$\hat{g}_B = \nabla L(w) + \epsilon \quad (1)$$

En vertu du théorème central limite, puisque les échantillons sont tirés indépendamment, la variance de ce bruit d'estimation est inversement proportionnelle à la taille du lot B . Si la variance du gradient pour une image individuelle est σ^2 , on obtient :

$$\text{Var}(\epsilon) \approx \frac{\sigma^2}{B} \quad (2)$$

L'équation de mise à jour des poids du réseau à l'itération k s'écrit donc :

$$w_{k+1} = w_k - \eta \hat{g}_B = w_k - \eta \nabla L(w_k) - \eta \epsilon \quad (3)$$

Donc le bruit, ou noise scale dans la littérature, est proportionnel à

$$\text{Noise Scale} \propto \frac{\eta}{B} \quad (4)$$

Pour le réduire, augmenter la batch size ou diminuer le learning rate par le même facteur ont donc le même effet.

2.1.3 Rétro-propagation

La rétro-propagation repose sur l'application itérative et astucieuse de la règle de la chaîne. Au lieu d'évaluer l'impact d'un poids de la première couche sur l'erreur finale de façon isolée (ce qui nécessiterait de recalculer tout le réseau pour chaque poids), l'algorithme calcule d'abord l'erreur à la couche de sortie, puis "propage" cette erreur vers l'arrière, couche par couche. Mathématiquement, considérons la valeur pré-activation d'un neurone j à la couche l , notée $z_j^{(l)} = \sum_k w_{jk}^{(l)} a_k^{(l-1)} + b_j^{(l)}$. L'impact d'un poids spécifique $w_{jk}^{(l)}$ sur le coût total C se calcule via la dérivée partielle :

$$\frac{\partial C}{\partial w_{jk}^{(l)}} = \frac{\partial C}{\partial z_j^{(l)}} \frac{\partial z_j^{(l)}}{\partial w_{jk}^{(l)}}$$

En remarquant que $\frac{\partial z_j^{(l)}}{\partial w_{jk}^{(l)}} = a_k^{(l-1)}$, l'équation se simplifie en :

$$\frac{\partial C}{\partial w_{jk}^{(l)}} = \delta_j^{(l)} a_k^{(l-1)}$$

où le terme $\delta_j^{(l)} = \frac{\partial C}{\partial z_j^{(l)}}$ représente "l'erreur" locale associée au neurone j de la couche l . La puissance computationnelle de la rétropropagation réside dans le fait que cette erreur $\delta_j^{(l)}$ peut être calculée de manière récursive et matricielle en fonction des erreurs de la couche suivante $\delta^{(l+1)}$, rendant le calcul du gradient global extrêmement efficace et adapté à un calcul sur GPU ou CPU.

2.1.4 Transition vers l'architecture convolutive (CNN)

Bien que cette approche MLP soit une excellente fondation théorique pour comprendre les réseaux de neurones, nous avons parallèlement exploré une approche alternative, CNN, que nous décrivons dans la section suivante. Et rapidement, nous nous sommes rendu compte que les résultats obtenus par cette seconde approche étaient beaucoup plus concluants. Même en testant différents hyperparamètres, en appliquant des transformations et essayant différentes optimisations, nous étions avec MLP bloqués en dessous de 87.5% de précision, là où rapidement l'équipe CNN a pu se rapprocher des 90%.

Et cette différence semble assez logique : les images du jeu de données EMNIST Balanced possèdent une structure spatiale inhérente sur une grille de 28×28 pixels. Aplatir ces images en vecteurs 1D de 784 éléments pour les introduire dans un MLP détruit la localité spatiale des informations, et la capacité de voir certaines formes spatiales, comme des boucles par exemple. Alors que CNN permet justement de reconnaître de tels patterns spatialement.

Nous avons donc décidé de concentrer nos efforts sur cette seconde approche, que nous décrivons ci-dessous.

2.2 Approche CNN

2.2.1 Introduction

Là où les MLP traitent les données comme un vecteur uni-dimensionnel, les réseaux de neurones convolutionnels (ou CNN en anglais) sont particulièrement adaptés aux images. Leur architecture permet d'exploiter la structure spatiale des données grâce à des opérations de convolution - d'où le nom - qui extraient des motifs, en réutilisant les mêmes poids sur toute l'image pour un même motif, baissant ainsi considérablement le nombre de poids nécessaires. En pratique, plusieurs filtres parcourent l'image, chacun détectant un motif particulier. Par exemple, un filtre pourrait apprendre à détecter, pour chaque zone de 3×3 pixels, une ligne verticale ou horizontale. Chaque filtre "glisse" sur l'image et attribue un score à chaque position, formant une carte de caractéristiques (feature map).

Pour réduire le nombre de paramètres tout en conservant l'information essentielle, on utilise souvent du pooling. Le max-pooling, par exemple, remplace chaque petite région par sa valeur maximale, ce qui réduit la taille des cartes de caractéristiques et rend le réseau plus robuste à de légères translations ou transformations.

Au fil des couches, les CNN apprennent des représentations de plus en plus complexes et abstraites en se basant sur les détections précédentes : les premières couches détectent des motifs simples, les couches intermédiaires se basent sur ces motifs pour reconnaître des formes plus complexes, et finalement la couche finale doit traduire les caractéristiques extraites de l'image en une prédiction de classe - des chiffres et lettres dans notre cas.

2.3 Techniques de test pour les hyperparamètres

Le CNN dépend de beaucoup d'hyperparamètres qui, bien choisis, peuvent considérablement augmenter son efficacité. Malheureusement, tester chacune des différentes combinaisons serait impossible, et il n'existe pas de consensus sur la valeur optimale de chacun de ceux-ci. Cependant, il est tout de même possible de se baser sur des standards existants pour certains paramètres, avant d'essayer empiriquement différentes valeurs afin de déterminer les valeurs idéales pour ce modèle.

Une première approche est de déterminer quels hyperparamètres on veut tester, et mener une recherche de grille (grid search). Cela revient à tester toutes les combinaisons possibles des hyperparamètres testés sur un espace discrétisé choisi. Cela garantit de trouver la solution optimale sur la grille, mais c'est extrêmement chronophage, sachant que l'entraînement complet de notre modèle nécessite environ une demi-heure.

Une autre approche est la recherche aléatoire (random search). Celle-ci revient à tester au hasard différentes combinaisons, et retenir la meilleure. Elle ne garantit pas d'obtenir la solution idéale, mais peut s'en approcher, tout en ayant un coût moindre.

Il serait intéressant d'avoir une méthode qui permet d'explorer plus en profondeur les zones prometteuses. C'est exactement ce que fait l'optimisation bayésienne. Cette approche construit un modèle probabiliste afin de déterminer quelles zones sont prometteuses. Elle peut donc à chaque itération soit aller dans une zone non explorée afin d'affiner son estimation, soit aller explorer dans une zone mieux connue, mais donnant des résultats satisfaisants. Cependant, cette méthode a encore un défaut : elle alloue un budget d'entraînement identique à chaque configuration, y compris à celles qui se révèlent rapidement peu performantes.

Dans un papier intitulé *Non-stochastic Best Arm Identification and Hyperparameter Optimization*[3], une autre méthode est proposée : le Successive Halving. Elle fonctionne de la manière suivante :

1. **Initialisation** On définit un budget $\mathbf{B}$ et un ensemble initial de n bras, représentant différentes configurations des hyperparamètres testés.
2. **Allocation et classification** Le budget $\mathbf{B}$ est réparti équitablement entre les différents bras. Ensuite, les différents bras sont entraînés avec le nombre d'époques allouées, et leur score (loss) est enregistré.
3. **Élimination et boucle** La moitié des bras ayant obtenu le moins bon score est éliminée, et la boucle recommence, avec le même budget total, mais sur deux fois moins de bras. Cette boucle continue jusqu'à ce qu'il ne reste plus qu'une configuration : la meilleure.

Cependant, cette méthode perd totalement certaines configurations "diesel" qui seraient plus lentes au démarrage, mais se stabiliseraient avec un meilleur résultat. Un autre papier, *Hyperband : A Novel Bandit-Based Approach to Hyperparameter Optimization*[4], publié deux ans plus tard, propose une solution : Il ne fait pas un unique cycle de successive halving, mais plusieurs, en partant d'une configuration très agressive (beaucoup de modèles à tester, peu de budget par modèle donc initialement) pour aller vers une très conservatrice (peu de modèles à tester et un budget plus important pour chacun donc). Le "champion" de chaque cycle de Hyperband est gardé, et le meilleur est donc la configuration choisie. Cela permet de retenir des configurations plus lentes à démarrer, mais qui convergeraient plus tard.

On peut tout de même regretter que Hyperband reparte de zéro entre chaque cycle, et n'utilise pas son expérience pour sélectionner quels points seraient intéressants à tester. C'est cela que recherche à améliorer un troisième papier : *BOHB : Robust and Efficient Hyperparameter Optimization at Scale*[5]. Il propose de reprendre le meilleur des deux mondes. Il s'inspire de l'approche de Hyperband, mais au lieu de sélectionner au hasard les configurations, il se base sur un modèle probabiliste, en trouvant un équilibre entre exploration et intensification. Ce dernier modèle nous a semblé l'approche la plus complète, avec un juste compromis entre l'approche bayésienne et d'Hyperband.

Cependant, nous nous sommes rendu compte qu'à l'échelle de notre projet, utiliser de telles méthodes semblait compliqué. A la place, nous avons testé individuellement chaque hyperparamètre, en faisant varier sa valeur tout en maintenant les autres constants. Ci-dessus, voici une analyse de nos essais pour chaque hyperparamètre.

2.3.1 Architecture des couches

La conception de l'architecture de notre réseau de neurones a nécessité de définir le nombre et le type de couches (convolutives ou denses), ainsi que le nombre de filtres pour chacune d'elles. En l'absence de règle universelle pour déterminer l'architecture idéale, nous avons opté pour une approche empirique. Notre méthodologie a consisté à entraîner plusieurs modèles sur 10 époques et à comparer leur précision finale. Afin de garantir l'objectivité des comparaisons, tous les autres hyperparamètres d'entraînement ont été maintenus constants. Bien que le critère de sélection principal fût la précision, nous avons également pris en compte le temps d'entraînement : un modèle très profond nécessitant trente minutes par époque n'est, par exemple, pas viable en pratique.

Toutes les architectures évaluées sont des variantes de notre premier modèle convolutif (soumis en S4), dont la configuration initiale est détaillée dans le tableau 1.

Étape	Opération
1	Convolution ($32 \times [28 \times 28]$)
2	Batch Normalization
3	ReLU
4	Max Pooling
5	Convolution ($64 \times [14 \times 14]$)
6	Batch Normalization
7	ReLU
8	Max Pooling
9	Linear (512)
10	ReLU
11	Dropout (0.5)
12	Linear (256)
13	ReLU
14	Dropout (0.5)
15	Linear (47)

TABLE 1 – Architecture du réseau de neurones initial

Ce modèle de base atteint une précision de 87,9 % après 10 époques. Pour tenter d'améliorer cette performance, nous avons exploré plusieurs pistes.

Première approche : augmentation de la profondeur du réseau.

En ajoutant un bloc convolutif supplémentaire à la fin de l'extracteur de caractéristiques (Convolution (128 filtres, 7×7) + Batch Normalization + ReLU + Max Pooling), la précision est montée à 88,5 %. L'ajout d'un second bloc supplémentaire (Convolution (256 filtres, 3×3) + Batch Normalization + ReLU + Max Pooling) a porté la précision à 88,6 %, ce qui ne représente qu'un gain marginal.

Deuxième approche : densification des blocs convolutifs.

En conservant l'architecture de base, nous avons inséré des couches de convolution supplémentaires entre les opérations de pooling. Avec deux couches de convolution successives avant chaque Max Pooling, la précision s'établit à 88,2 %. Bien que légère, cette évolution reste positive. En passant à trois couches de convolution successives par bloc, nous avons retrouvé une précision de 88,5 %.

Troisième approche : élargissement des couches.

Nous avons testé l'impact d'une augmentation significative du nombre de filtres dès les premières couches. En quadruplant ce nombre (128 filtres au lieu de 32 pour la première couche, puis 256 au lieu de 64 pour la seconde), la précision obtenue après 10 époques était de 87,8 %. L'augmentation stricte du nombre de neurones n'a donc pas eu d'effet bénéfique.

Quatrième approche : maintien d'un nombre constant de filtres.

Nous avons vérifié s'il était pertinent de garder un nombre constant de filtres tout au long des couches de convolution. Un test avec deux couches consécutives à 32 filtres a donné une précision de 87,8 %. Pour comparer équitablement avec notre première idée (croissance $32 \rightarrow 64 \rightarrow 128$), nous avons testé trois couches maintenues à 32 filtres. La précision a chuté à 88,1 % (contre 88,5 % pour le modèle croissant). Enfin, l'augmentation de la taille des batchs à 128 (au lieu de 32) sur cette structure n'a pas permis de dépasser 88,1 %.

Architecture finale

En combinant les approches 1 et 2, qui se sont révélées être les seules réellement efficaces, nous avons abouti à l'architecture finale présentée dans le tableau 2. Celle-ci atteint notre meilleure précision : 88,9 % après 10 époques.

Étape	Opération
1	Convolution ($32 \times [28 \times 28]$)
2	Batch Normalization
3	ReLU
4	Convolution ($32 \times [28 \times 28]$)
5	Batch Normalization
6	ReLU
7	Max Pooling
8	Convolution ($64 \times [14 \times 14]$)
9	Batch Normalization
10	ReLU
11	Convolution ($64 \times [14 \times 14]$)
12	Batch Normalization
13	ReLU
14	Convolution ($128 \times [7 \times 7]$)
15	Batch Normalization
16	ReLU
17	Convolution ($128 \times [7 \times 7]$)
18	Batch Normalization
19	ReLU
20	AdaptiveAvgPool2d(3)
21	Linear (512)
22	ReLU
23	Dropout (0.5)
24	Linear (256)
25	ReLU
26	Dropout (0.5)
27	Linear (47)

TABLE 2 – Architecture du réseau de neurones finale

Finalement, pour notre architecture finale, nous avons remplacé la toute dernière couche de Max Pooling par une couche AdaptiveAvgPool2d(3). Contrairement au Max Pooling classique, qui ne conserve que la valeur maximale d'une zone au risque de perdre des informations, le AdaptiveAvgPool2d condense l'information de manière globale sur la zone ciblée. Nos tests ont confirmé que cette approche offrait de meilleurs résultats, que ce soit sur des images saines ou attaquées.

2.3.2 Le taux d'apprentissage

Le taux d'apprentissage (learning rate) est un hyperparamètre clé qui contrôle la taille des pas effectués lors de l'optimisation. Afin de choisir une valeur adaptée, nous avons testé plusieurs learning rates en l'absence de scheduler. Pour chaque valeur, nous avons entraîné le modèle et retenu la meilleure performance atteinte en termes d'accuracy sur l'ensemble de test.

Cette approche permet d'isoler l'impact du learning rate et d'identifier un bon compromis : une valeur trop faible ralentit la convergence, tandis qu'une valeur trop élevée peut rendre l'entraînement instable. Le learning rate retenu est donc 10^{-3} qui maximise l'accuracy finale du modèle 1.

2.3.3 Le scheduler de ce taux d'apprentissage

Une fois le learning rate initial fixé, nous avons étudié différentes stratégies de scheduler, c'est-à-dire des méthodes permettant de faire évoluer ce taux au cours de l'entraînement.

Nous avons comparé deux approches principales :

— **StepLR(step_size,gamma)** :

- **step_size** : nombre d'epochs après lequel le learning rate est réduit. Plus cette valeur est grande, plus la décroissance est lente.
- **gamma** : facteur multiplicatif appliqué au learning rate à chaque étape. Par exemple, une valeur de 0.5 divise le learning rate par deux à chaque **step_size**.

— **ReduceLROnPlateau(factor,patience,threshold)** :

- **factor** : facteur de réduction du learning rate lorsque la métrique cesse de s'améliorer. Par exemple, 0.5 réduit le learning rate de moitié.
- **patience** : nombre d'epochs à attendre sans amélioration avant de déclencher la réduction du learning rate. Une patience faible rend le scheduler plus réactif.
- **threshold** : seuil minimal d'amélioration considéré comme significatif. Si l'amélioration est inférieure à ce seuil, elle est ignorée, ce qui évite des déclenchements trop fréquents.

Après comparaison des différentes configurations du scheduler Figure 2, nous avons retenu le scheduler **ReduceLROnPlateau** avec les paramètres $factor = 0.2$, $patience = 2$ et $threshold = 10^{-2}$. Ce choix s'explique par leur capacité à maintenir un scheduler véritablement dynamique, tout en offrant les meilleures performances parmi les configurations testées.

En effet, ces paramètres permettent une adaptation efficace du learning rate en fonction de l'évolution de la loss, sans provoquer de réduction trop précoce qui pourrait freiner l'apprentissage. Les autres configurations testées se sont révélées soit trop conservatrices (réduction tardive du learning rate), soit trop agressives (réduction trop rapide), ce qui limitait les performances globales.

Par ailleurs, l'analyse des courbes d'apprentissage sur 30 epochs montre que, malgré des dynamiques différentes entre **StepLR** et **ReduceLROnPlateau**, les modèles convergent vers des niveaux d'accuracy finaux très similaires. Cela suggère que, à horizon fixé, le choix du scheduler influence principalement la trajectoire d'apprentissage (vitesse de convergence et stabilité), mais a un impact limité sur la performance finale.

Ainsi, le choix de $factor = 0.2$ et $threshold = 10^{-2}$ pour **ReduceLROnPlateau** constitue un compromis pertinent entre adaptabilité et performance, tout en exploitant pleinement le caractère dynamique de ce type de scheduler.

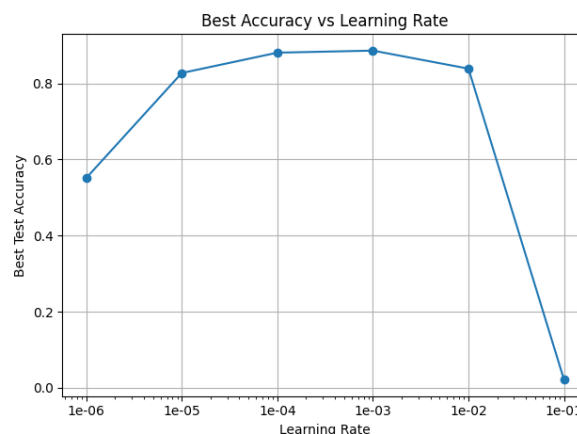


FIGURE 1 – Influence du learning rate sur l'accuracy (sans scheduler)

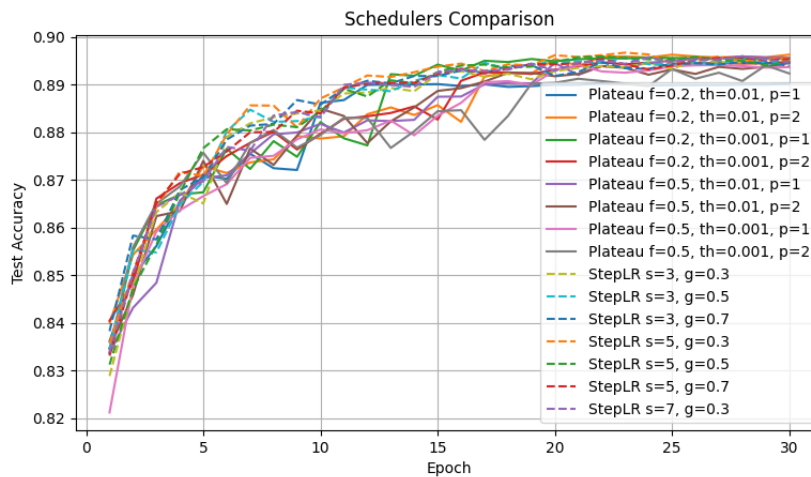


FIGURE 2 – Comparaison des schedulers pour un learning rate fixé à 10^{-3} (f=factor, th=threshold, p=patience, s=step, g=gamma)

2.3.4 Batch size et nombre d'époques

La batch size, ou taille de lot, est le nombre d'image sur lesquelles on calcule notre gradient avant de procéder à un ajustement des poids. Il est important de choisir une taille adaptée à notre modèle, dataset et entraînement afin d'avoir des performances optimales.

Un epoch est un cycle d'entraînement sur toutes les données. Le nombre de changements de poids par epoch dépend donc de la batch size : c'est la division de la taille du dataset par le batch size.

Si on choisit un très petit batch size, on risque d'avoir beaucoup de bruit, car la taille du lot ne permet pas d'avoir une image fidèle du gradient réel. De plus, le temps pour entraîner une seule époque sera important, vu les nombreux changements de gradient intermédiaires, sans que ce ne soit forcément reflété dans les performances. Néanmoins, une petite batch size permet d'éviter de tomber dans un minimum local, comme le gradient est bien plus aléatoire.

De l'autre côté, un grand batch size permet de lisser la descente : il est une réflexion plus juste du gradient réel sur toutes les données. Cependant, cela peut aussi bloquer le modèle dans un minimum local. De plus, un plus grand batch requiert beaucoup de mémoire. En effet, toutes les valeurs intermédiaires des couches doivent être conservées pour pouvoir effectuer la rétro-propagation. Il faut donc veiller à ce que l'ordinateur puisse assumer cette charge. Mais le temps d'exécution, grâce aux changements moins fréquents de poids, est moindre avec un batch plus grand. Finalement, il semble conventionnel, même si non obligatoire, d'avoir un exposant de 2 comme batch size. Nous adopterons cette convention, qui peut servir pour le gpu et la mémoire (qui fonctionnent eux aussi en puissances de 2), même si l'article suivant [6] semble douter de la nécessité de telles contraintes.

Le papier suivant, intitulé *"Revisiting Small Batch Training for Deep Neural Networks"* (Masters & Lusch, 2018), semble suggérer que des batch sizes très petits, entre 2 et 32, offrent des performances supérieures. Et en effet, lorsque nous avons testé différentes tailles de batch entre 16 et 512 (puissances de 2) sur 15 époques (3), les performances finales étaient de plus en plus dégradées, de 89 à 86% avec l'agrandissement de la taille de lot. Cependant, le temps d'entraînement, de 512 s pour un lot de 32, descendait à 231 s pour celui de 512. Mais sur un nombre fixe d'époques, et avant convergence, il est logique que les petites tailles de lot performant mieux : elles ont eu plus d'opportunité de corriger leurs poids. Vu le temps d'exécution fort important, nous n'avons testé sur 15 époques, mais pas jusqu'à convergence pour les 5 poids précédemment testés. Mais nous avons ensuite re-lancé un entraînement, cette fois sur 30 époques, avec trois tailles : 32, 64 et 128 4. C'est à nouveau 32 qui a fourni une meilleure précision, mais avec une avance très faible de 0.05% d'accuracy. Par contre, le temps d'exécution sur les 30 époques passe de 1434 à 1002 secondes. Ensuite de 64 à 128 le temps d'exécution ne varie plus énormément (937s pour 128). Nous avons donc choisi de sélectionner une taille de lot de 64, sacrifiant potentiellement une très légère performance, mais permettant de gagner un temps significatif lors de l'entraînement. De plus,

nous avons mené ce test sur 30 époques, mais potentiellement sur un entraînement plus long l'écart aurait été résorbé.

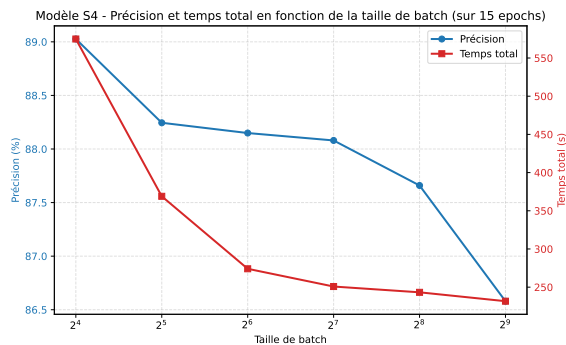


FIGURE 3 – Précision et temps d'exécution en fonction du batch size sur 15 époques

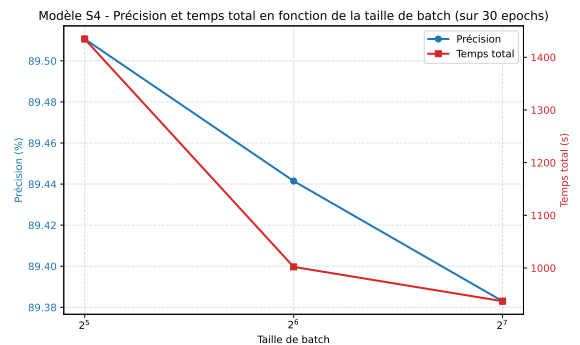


FIGURE 4 – Précision et temps d'exécution en fonction du batch size sur 30 époques

Pour le nombre d'époques, puisque nous changeons de modèle uniquement si l'époque a produit un nouveau meilleur modèle, réaliser plus d'époques ne semble pas pouvoir détériorer les performances. Néanmoins, nous avons une contrainte de temps, et après un certain temps les gains semblent limités (même si un phénomène de grokking pourrait se produire, propulsant soudainement, après une période de "désert d'améliorations", les performances de notre modèle). Nous avons d'abord pensé à compter le nombre d'époques passées depuis la dernière amélioration de la précision. Si ce compteur dépasse une certaine patience que nous avons fixée à 5 (subjectivement, il est difficile de tester différentes valeurs pour ce paramètre, puisqu'il demande de faire tourner notre modèle jusqu'à une convergence complète), ou si le nombre d'époques maximum était dépassé (que nous avons fixé à 70) nous arrêtons notre entraînement. Cependant, nous avons finalement décidé de fixer, sur base de nos observations, le nombre d'époques à 41 sur notre modèle final.

2.4 Performances initiales

Finalement, notre modèle atteint une précision de 89,5% sur l'ensemble du jeu de données. Concrètement, cela signifie qu'une erreur de reconnaissance survient environ une fois sur dix. Si ce résultat peut paraître insuffisant pour des applications exigeant une sécurité élevée, il représente néanmoins une performance solide.

En effet, ce score est encourageant lorsqu'on le compare aux meilleurs résultats mondiaux obtenus sur le jeu de données EMNIST Balanced. À titre de comparaison, le modèle WaveMix-Lite, qui fait référence dans ce domaine, affiche une précision de 91,06% [7]. Le fait que notre modèle se situe à moins de 2% d'écart de cette référence mondiale démontre l'efficacité de notre approche face à la complexité de ce dataset, qui mélange des chiffres et des lettres manuscrits avec donc certaines classes qui sont naturellement ambiguës (par exemple *l* et *I*, ou encore *O* et *0*), ce qui introduit une incertitude intrinsèque indépendante du modèle utilisé.

3 Phase 2 : Stratégies d'attaque adversariale

3.1 Contraintes et méthodologie

L'objectif de l'attaquant est de construire, pour une image d'entrée x correctement classifiée par un modèle cible f , une perturbation δ la plus petite possible qui induit une erreur de classification. Ce problème se formalise comme un problème d'optimisation non-convexe :

$$\max_{\delta} \mathcal{L}(f(x + \delta), y) \quad \text{s.t.} \quad \|\delta\|_2 \leq \varepsilon, \quad x + \delta \in [0, 1]^{28 \times 28}$$

où y est la vraie classe, $\mathcal{L}$ est une fonction de perte à maximiser, et ε est le rayon maximal de perturbation autorisé. La contrainte est une boule euclidienne (norme L_2) : cela implique que la perturbation est limitée en énergie globale, et non pixel par pixel. En pratique, on peut calculer le gradient par rapport

aux pixels d'entrée sans connaître l'architecture ni les poids. Dans l'espace $\mathbb{R}^{784}$, l'image x se trouve dans une région où le classifieur répond correctement. La frontière de décision sépare cette région des régions correspondant aux autres classes. L'objectif de l'attaque est de trouver le point $x + \delta$ le plus proche de x (au sens L_2) qui franchit cette frontière, c'est-à-dire pour lequel $\arg \max f(x + \delta) \neq y$.

3.2 Approche d'attaque n°1 : PGD+

Pour notre première attaque, nous avons choisi d'implémenter une version améliorée de l'attaque PGD (*Projected Gradient Descent*) madry2018. Le principe de base consiste à modifier itérativement l'image dans la direction qui maximise la perte du modèle, tout en projetant la perturbation sur une boule L_2 de rayon ε pour rester dans le budget autorisé. La règle de mise à jour à chaque itération t est :

$$\delta^{(t+1)} = \Pi_\varepsilon \left[\delta^{(t)} + \alpha \cdot \frac{\nabla_\delta \mathcal{L}(f(x + \delta^{(t)}), y)}{\|\nabla_\delta \mathcal{L}(f(x + \delta^{(t)}), y)\|_2} \right]$$

où α est le pas de gradient et $\Pi_\varepsilon(\delta) = \delta \cdot \min\left(1, \frac{\varepsilon}{\|\delta\|_2}\right)$ est la projection sur la boule L_2 . Un point d'implémentation important : la projection doit être appliquée *après* le clamp de l'image dans $[0, 1]$, et non avant, afin de ne pas produire d'images invalides. L'ordre correct est donc : calculer $x_{\text{adv}} = \text{clamp}(x + \delta, 0, 1)$, recalculer $\delta = x_{\text{adv}} - x$, puis projeter δ sur la boule L_2 .

3.2.1 Améliorations apportées à PGD

PGD de base souffre de deux limitations principales : il peut rester bloqué dans un maximum local selon son point de départ, et son pas fixe n'est pas adapté à tous les paysages de loss. Nous avons donc ajouté quatre améliorations.

3.2.1.1 Momentum. Inspiré de Dong et al. (2018), le momentum accumule la direction du gradient entre les itérations, lissant la trajectoire dans l'espace des perturbations :

$$g^{(t+1)} = \mu \cdot g^{(t)} + \frac{\nabla_\delta \mathcal{L}(f(x + \delta^{(t)}), y)}{\|\nabla_\delta \mathcal{L}\|_2}, \quad \delta^{(t+1)} = \Pi_\varepsilon \left[\delta^{(t)} + \alpha \cdot g^{(t+1)} \right]$$

avec $\mu = 0.9$. Nous utilisons le vecteur $g^{(t+1)}$ directement (et non son signe) afin de conserver l'information de magnitude accumulée.

3.2.1.2 Diversification des fonctions de perte. Plutôt que d'utiliser une seule fonction de perte sur tous les restarts, nous en alternons trois, chacune explorant une géométrie différente de l'espace des logits :

- **Loss Margin** : $\max_{j \neq y} z_j - z_y$. Simple et sans saturation, mais sensible à l'échelle des logits.
- **DLR non-ciblée** [8] : $-\frac{z_y - \max_{j \neq y} z_j}{z_y - z_{\pi_3}}$, où z_{π_3} est le troisième logit par ordre décroissant. Le dénominateur normalise l'écart par rapport à la marge de confort du classifieur, ce qui rend le gradient comparable entre des modèles de calibrations différentes.
- **DLR ciblée** : $-\frac{z_{t^{(t)}} - z_y}{|z_y - z_{\pi_3}|}$, où $t^{(t)} = \arg \max_{j \neq y} z_j^{(t)}$ est le *challenger* recalculé à chaque itération. C'est l'apport principal de notre implémentation : plutôt que de fixer la classe cible sur l'image originale, on la réévalue à chaque étape, ce qui permet de suivre la frontière de décision au fur et à mesure que la perturbation évolue. Nos expériences montrent que cette variante est la plus efficace, en particulier à faible ε .

Le tableau 3 résume les différences entre un challenger fixe et dynamique.

Propriété	Challenger fixe (x_{orig})	Challenger dynamique ($x_{\text{adv}}^{(t)}$)
Cible	Figée au départ	Suit la trajectoire
Efficacité à faible ε	Réduite	Maximale
Gaspillage de gradient	Oui	Non

TABLE 3 – Comparaison entre challenger fixe et dynamique

3.2.1.3 Pas adaptatif. Lorsque la loss stagne pendant 5 itérations consécutives (variation inférieure à 10^{-6}), le pas est réduit selon :

$$\alpha^{(k+1)} = \max\left(\frac{\alpha^{(k)}}{2}, \alpha_0 \cdot 0.05\right)$$

Le plancher $\alpha_0 \times 0.05$ évite que le pas ne devienne numériquement nul. Le pas initial est fixé à $\alpha_0 = \varepsilon \times 2.5/n_{\text{iter}}$, ce qui permet d'explorer une région de taille 2.5ε en T itérations.

3.2.1.4 Multi-restart. PGD étant non-convexe, il converge vers un maximum local dépendant du point de départ. Nous lançons donc $R = 6$ restarts depuis des initialisations distinctes et retenons la perturbation la plus efficace. Si chaque restart a une probabilité p de succès, la probabilité qu'au moins un réussisse est $1 - (1 - p)^R$. Avec $R = 6$ et $p = 0.4$, cela donne environ 95% de succès, ce qui représente un bon compromis avec la contrainte de 10 secondes par image.

Chaque initialisation tire un point uniformément dans le *volume* de la boule L_2 (et non sur sa surface) selon :

$$u \sim \mathcal{N}(0, I_{784}), \quad \hat{u} = \frac{u}{\|u\|_2}, \quad r \sim \text{Uniform}(0, \varepsilon)^{1/784}, \quad \delta^{(0)} = r \cdot \hat{u}$$

La puissance $1/784$ est essentielle pour éviter que tous les points soient concentrés près de la surface de la boule, ce qui assure une bonne diversité entre les restarts.

3.2.2 Résultats

Le tableau suivant présente le résultat des contributions de chaque amélioration, mesurée sur 300 images.

Méthode	$\varepsilon = 0.1$	$\varepsilon = 0.3$	$\varepsilon = 0.5$	$\varepsilon = 1.0$	Temps/img
PGD standard (T=50, R=1)	7.7%	20%	41%	89%	~ 2 s
+ Momentum	7.7%	19%	39.3%	89%	~ 2 s
+ DLR non-ciblée	8%	19.3%	38%	80%	~ 2 s
+ Multi-start R=6	10%	23%	43%	92%	~ 6 s
PGD+ complet (soumis)	10%	22.3%	41%	90%	~ 7 s

La contribution la plus significative est le multi-restart, qui améliore les résultats à tous les niveaux de ε . Le momentum et la DLR non-ciblée contribuent surtout à la stabilité de la convergence. Bien que la configuration "+ Multi-start R=6" présente déjà de bonnes performances, nous avons retenu la version PGD+ complet pour sa capacité à éviter des pièges classiques, notamment celui de la stagnation (plateaux de gradient). En adaptant le pas de gradient et en réévaluant dynamiquement la cible, le PGD+ garantit une convergence plus fiable là où une version simplifiée pourrait rester bloquée.

3.2.3 Analyse de cas concrets

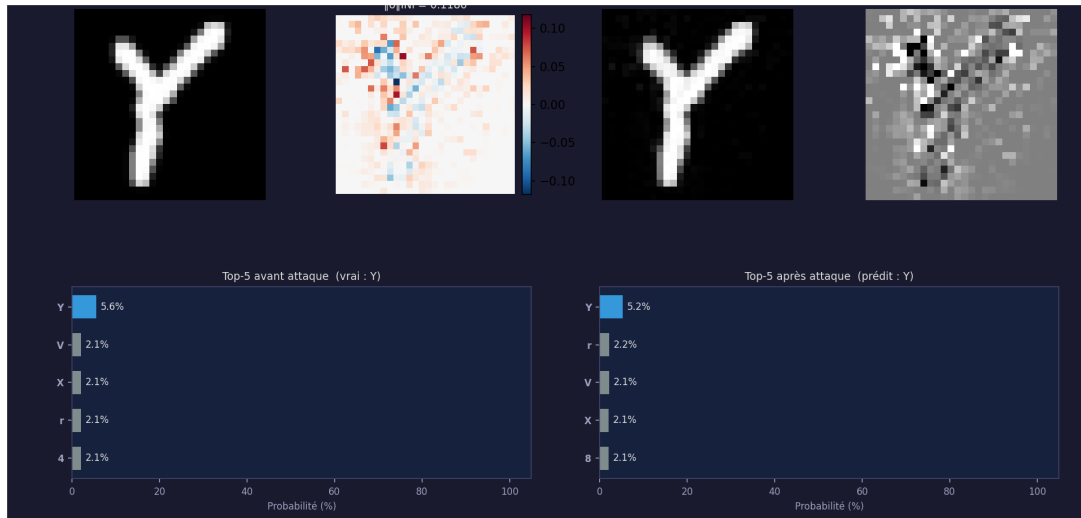


FIGURE 5 – Attaque PGD+ ($\epsilon = 0.5$) sur une image 'Y' : échec. De gauche à droite : image originale, perturbation δ (rouge = augmenté, bleu = diminué), image attaquée, perturbation amplifiée $\times 10$. En bas : distribution Top-5 des probabilités avant et après attaque.

3.2.3.1 Cas 1 : échec sur 'Y'. La confiance initiale du modèle sur 'Y' n'est que de 5.6%, à peine au-dessus de la distribution uniforme sur 47 classes (2.1%). Après l'attaque, il n'y a quasiment aucun changement dans les probabilités Top-5, comme on peut le voir sur la figure 5. L'attaque a consommé la totalité du budget sans réussir à franchir la frontière de décision.

Ce cas illustre une limite fondamentale des attaques par gradient : lorsque le modèle est déjà très incertain, il n'existe pas de frontière nette à franchir dans le rayon ϵ . Trois facteurs expliquent cet échec : la distribution initiale quasi-uniforme laisse peu de gradient à exploiter, le paysage de loss est plat car tous les logits sont proches, et la lettre 'Y' partage des caractéristiques morphologiques avec V, X et r, ce qui rend l'ambiguïté inhérente à cette image.



FIGURE 6 – Attaque PGD+ ($\epsilon = 0.5$) sur une image '1' : succès.

3.2.3.2 Cas 2 : succès sur '1'. Le modèle prédit initialement '1' avec 3.6% de confiance, 'L' étant déjà en deuxième position à 2.9%, soit un écart de seulement 0.7 point de pourcentage. Après l'attaque, 'L' passe devant '1' (3.5% vs 2.9%). Ce cas réunit les conditions idéales : une frontière de décision à faible distance, et une ambiguïté morphologique réelle — un '1' manuscrit penché ressemble visuellement à un

'L' ou un 'I'. La DLR ciblée concentre tout le budget dans la direction qui maximise $z_L - z_1$, ce qui suffit à faire basculer la prédiction. On notera que '1' reste dans le Top-5 après l'attaque : la perturbation n'a pas effacé sa signature visuelle, elle a simplement légèrement déplacé la balance.

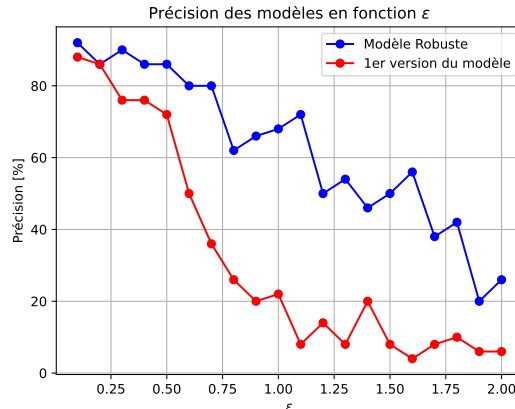


FIGURE 7 – Précision du modèle initial et du modèle robuste en fonction de ϵ (20 images testées par valeur de ϵ).

On peut voir sur la figure 7 qu'un ϵ de 0.7 suffit à réduire la précision du modèle initial de moitié environ, ce qui confirme sa vulnérabilité aux attaques adversariales. Le modèle robuste résiste nettement mieux, atteignant encore 80% de précision pour cette même valeur.

3.3 Approche d'attaque n°2

Pour la seconde attaque, nous avons décidé de changer d'approche et de résoudre un problème d'optimisation. Pour la formulation du problème d'optimisation, nous nous sommes basés sur l'attaque venant du papier *Towards Evaluating the Robustness of a Neural Network*[9] dont la formulation initiale est la suivante :

$$\begin{aligned} &\text{minimiser } \mathcal{D}(x, x + \delta) \\ &\text{sous la contrainte que } C(x + \delta) \neq t \\ &\text{Et } x + \delta \in [0, 1]^{28 \times 28} \end{aligned}$$

Dans le cadre de notre projet, $\mathcal{D}$ est la norme l_2 de la perturbation δ et $C(x)$ nous renvoie la plus grande probabilité de la classe de l'image x . La fonction $C(x)$ étant non-linéaire, nous allons choisir une formulation équivalente qui sera plus approprié pour les algorithmes d'optimisation déjà existant. Plusieurs pistes ont été évalué dans le papier, nous choisissons directement celle avec les meilleurs résultats selon le papier qui est la suivante :

$$f(x') = \max(\max\{F(x')_i : i \neq t\} - F(x')_t, -\kappa)$$

Avec $F(x')$ qui nous rend les probabilités que x' appartiennent à chaque classe¹ et κ est un paramètre qui permet de contrôler à quel point on est confiant sur la mis-classification de l'image. L'intuition derrière le choix de la fonction $f(x')$ est que $f(x') \leq 0$ si et seulement si $\max\{F(x')_i : i \neq t\} - F(x')_t \leq 0$, c'est-à-dire que le modèle suggère que x' a une plus grande probabilité d'appartenir à une classe i au lieu de sa réelle classe t , cela veut donc dire qu'on a réussi à modifier la classification de l'image. On ajoute également une constante c à notre problème pour réduire le nombre de contrainte, nous obtenons la formulation suivante :

$$\begin{aligned} &\min_{\delta \in \mathbb{R}^{28 \times 28}} \|\delta\|_2 + c \cdot f(x + \delta) \\ &\text{sous la contrainte que } x + \delta \in [0, 1]^{28 \times 28} \end{aligned}$$

1. En pratique, dans le papier, ils utilisent les logits plutôt que la fonction qui retourne le vecteur de probabilités mais nous n'avons pas eu de différences importantes en terme de performance en utilisant ce dernier.

Il nous reste désormais une dernière contrainte qui est que l'image attaquée doit rester une image valide, c'est-à-dire que nous devons avoir $0 \leq x_i + \delta_i \leq 1 \forall i \in \{1, \dots, 28\}$. L'approche qui a été retenue dans le papier est celle d'effectuer le changement de variable et d'optimiser selon la variable w_i selon l'équation suivante :

$$\delta_i = \frac{1}{2}(\tanh(w_i) + 1) - x_i$$

Avec ce changement de variable, nous sommes moins restreint sur le choix de l'algorithme d'optimisation à utiliser car notre problème est désormais sans contraintes. En combinant ce dernier résultat avec ceux précédant, on obtient la formulation complète suivante :

$$\min_{w \in \mathbb{R}^{28 \times 28}} \left\| \frac{1}{2}(\tanh(w) + 1) - x \right\|_2^2 + c \cdot f\left(\frac{1}{2}(\tanh(w) + 1)\right)$$

La formulation ci-dessus ne prends pas en compte la contrainte sur la norme de la perturbation qui nous est imposé dans le cadre du projet. Pour remédier à ce problème, après convergence de l'algorithme, on projette la perturbation sur la boule de rayon ϵ selon la norme l_2

Dans l'attaque décrite ci-dessus, nous avons 2 paramètres à choisir, c et κ . Un choix optimale de la valeur de c , trouvé empiriquement, est de choisir la valeur la plus petite de c tel que $f(x') \leq 0$ car, lors de la descente de gradient, les 2 termes de la fonction objective seront minimiser simultanément à la place de minimiser un terme avant l'autre. Donc pour choisir c , nous devons effectuer une recherche dichotomique mais en pratique dans notre implémentation, nous choisirons $c = 1$ car $c > 1$ rend l'attaque moins efficace et $c < 0.1$ rends la convergence de l'attaque beaucoup plus difficile à atteindre. Concernant la valeur de κ , pour notre implémentation, nous choisissons $\kappa = 0$ pour les raisons détaillées dans le papier.

3.3.1 Analyse des performances

Nous avons désormais déterminé tout les paramètres du problèmes, on va pouvoir analyser les performances de notre attaque à l'aide des graphes ci-dessous.



FIGURE 8 – L'image à gauche est l'image initiale, celle du milieu représente une carte thermique de la perturbation pour un $\epsilon = 0.9$ et l'image de droite est l'image perturbée

Tout d'abord, on peut voir sur la figure [8] que l'attaque est quasiment imperceptible à l'œil nu ce qui est crucial dans le cadre de notre projet. En examinant la carte thermique de la perturbation, on peut voir qu'on augmente la valeur des pixels situé hors du caractère et on diminue la valeur des pixels présent dans le caractère. On peut donc conclure que l'algorithme aura tendance à essayer d'inverser les valeurs des pixels dans une image.

Nous allons également vérifier l'efficacité de cette seconde attaque en réalisant un graphe sur la précision de notre premier modèle en fonction de la valeur de ϵ .

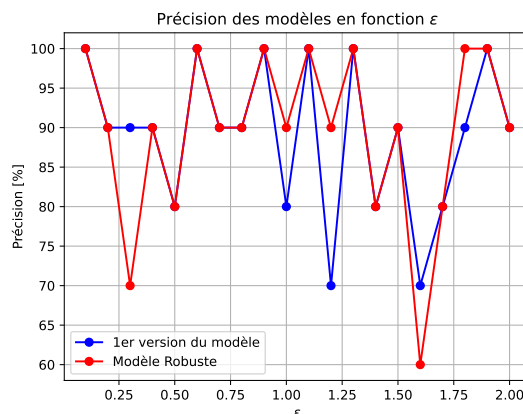


FIGURE 9 – Précision du modèle initial et du modèle robuste en fonction de ϵ (20 images ont été testé pour chaque valeur de ϵ)

On peut voir sur la figure [9] qu'il n'y a pas de différence importante entre la précision de la première version du modèle et notre modèle plus robuste (les détails des améliorations sont décrites dans la section [4]). On peut également apercevoir que la précision des 2 modèles vaut en moyenne 90 %, ce qui veut dire que l'attaque a un effet presque négligeable pour altérer la précision des 2 modèles. La raison du manque de performance de l'attaque est dû au fait qu'on doit respecter une contrainte sur la norme de la perturbation alors que le but même de l'attaque est d'inverser au maximum la valeur des pixels donc la contrainte diminue considérablement l'efficacité de l'attaque.

3.4 Comparaison entre nos deux attaques

3.4.1 Formulations mathématiques comparées

Les deux méthodes résolvent des problèmes d'optimisation opposés.

PGD+	C&W
$\max_{\delta} L(f(x + \delta), y)$	$\min_w \ x_{\text{adv}} - x\ _2^2$
$\text{s.t. } \ \delta\ _2 \leq \epsilon \text{ (budget fixé)}$	$+ c \cdot f(x_{\text{adv}})$
$x + \delta \in [0, 1]$	

On se donne ϵ et on cherche la perturbation la plus efficace à l'intérieur de ce budget.

On cherche la perturbation la plus petite possible qui trompe le modèle.

L'opposition fondamentale : PGD+ part d'un budget ϵ imposé et cherche à l'exploiter au maximum. C&W ne fixe pas de budget il le découvre par l'optimisation, en cherchant le minimum de $\|\delta\|_2$ suffisant pour franchir la frontière de décision.

3.4.2 Mécanismes : comment chaque méthode construit δ

PGD+ : montée de gradient itérative avec projection

À chaque itération, PGD+ fait un pas dans la direction du gradient de la loss, puis projette δ sur la boule L_2 pour rester dans le budget ϵ :

$$g^{(t+1)} = \mu \cdot g^{(t)} + \frac{\nabla_{\delta} L}{\|\nabla_{\delta} L\|_2} \quad (\text{momentum})$$

$$\delta^{(t+1)} = \Pi_{\epsilon} \left[\text{clamp}(x + \delta^{(t)} + \alpha \cdot g^{(t+1)}, 0, 1) - x \right]$$

Le gradient est calculé sur tous les 784 pixels simultanément. Chaque pixel reçoit une modification proportionnelle à son gradient, indépendamment de sa valeur courante. Un pixel noir du fond peut avoir un grand gradient si ses poids dans le réseau sont importants — PGD+ le modifiera.

C&W : optimisation de boîte noire via Adam + changement de variable

C&W introduit une variable intermédiaire w et pose :

$$x_{\text{adv}} = \frac{1}{2} \cdot (\tanh(w) + 1)$$

Ce changement de variable est central : puisque $\tanh$ mappe $\mathbb{R} \rightarrow (-1, 1)$, on a automatiquement $x_{\text{adv}} \in (0, 1)$ pour n'importe quelle valeur de w , sans jamais faire de clamp ou de projection. On peut alors optimiser w librement avec Adam. La loss totale minimisée est :

$$L_{\text{total}} = \|x_{\text{adv}} - x\|^2 + c \cdot f(x_{\text{adv}})$$

Chaque terme joue un rôle précis :

- $\|x_{\text{adv}} - x\|^2$: minimise la distance L_2 entre l'image attaquée et l'originale — c'est l'objectif de petite perturbation.
- $c \cdot f(x_{\text{adv}})$: force le modèle à se tromper. La fonction f mesure l'écart entre le logit de la vraie classe et le meilleur logit concurrent. c est un hyper-paramètre critique qui équilibre les deux objectifs.

3.4.3 Observation clé

Sur la carte C&W ([8]), les contours de couleur suivent exactement le tracé du '9' les zones jaune/orange/rouge bordent précisément les pixels blancs. Le fond jaune pâle indique une perturbation quasi-nulle (~ 0). Sur une carte PGD+ équivalente, le bruit serait réparti sur l'ensemble de l'image, fond inclus.

PGD+ modifie le fond : pourquoi ?

PGD+ calcule le gradient $\partial L / \partial \delta$ sur tous les pixels et fait un pas dans cette direction. Un pixel noir du fond vaut 0.0 — sa contribution courante au neurone est $0.0 \times w = 0$. Mais si son poids w dans le réseau est grand (ce qui arrive quand le modèle a appris des corrélations statistiques entre le fond et une classe), alors :

$$\frac{\partial L}{\partial \delta}[r, c] = w_{r,c} \times (\text{signal rétropropagé})$$

Ce gradient peut être grand même si le pixel vaut 0. PGD+ va donc modifier ce pixel du fond — non pas par choix délibéré, mais parce que la montée de gradient est aveugle à la valeur courante du pixel : elle ne regarde que le gradient.

De plus, PGD+ consomme toujours la totalité du budget ε . La projection L_2 ramène δ exactement sur la sphère de rayon ε , répartissant l'énergie sur tous les pixels modifiés fond inclus. Il n'y a aucun mécanisme qui inciterait PGD+ à ne pas modifier le fond.

La perturbation de C&W suit les contours : pourquoi ?

Sur la figure [8], on observe que la perturbation C&W est maximale sur les bords du tracé, et non au centre des pixels blancs. C'est une conséquence directe de la géométrie du problème. Les pixels en bord de tracé sont ceux pour lesquels une petite modification change le plus la classification : ils se trouvent à la frontière entre "tracé" (blanc) et "fond" (noir), là où la sensibilité du réseau est la plus grande. Modifier ces pixels de bord déplace efficacement la frontière perçue du chiffre — une légère modification du bord supérieur du '9' peut le faire ressembler à un autre chiffre.

En termes de gradient, les pixels de bord ont à la fois une grande valeur de gradient (ils contribuent activement à la décision) et un faible coût de modification (ils ne sont pas très blancs, donc $(x_{\text{adv}}[r, c] - x[r, c])^2$ reste petit même avec une modification sensible). C&W concentre naturellement son énergie là.

4 Phase 3 : Le classifieur robuste

4.1 Techniques de défense et stratégies de robustesse

4.1.1 Entraînement adversarial de base

La méthode de défense la plus directe consiste à intégrer la menace adversariale directement durant la phase d'entraînement du modèle. L'utilisation d'attaques itératives complexes a été envisagée dans un

premier temps, mais le temps d'exécution de ces méthodes était trop long pour être appliqué entièrement sur le dataset d'entraînement. C'est pourquoi une technique générant des exemples adversariaux en une seule étape a été privilégiée.

Plutôt que d'entraîner le réseau exclusivement sur des images propres, on l'optimise pour qu'il traite correctement les images perturbées. En pratique, pour chaque lot d'images X , des images adversariales X_{adv} sont construites à partir d'une version normalisée du gradient. La perturbation δ , contrainte par une norme L_2 , est calculée comme suit :

$$\delta = \epsilon \cdot \frac{\nabla_x \mathcal{L}(f(x), y)}{\|\nabla_x \mathcal{L}(f(x), y)\|_2 + \eta} \quad (5)$$

Dans cette formule, ϵ représente l'amplitude de l'attaque (le budget de perturbation) et $\eta = 10^{-8}$ est un terme de stabilité numérique destiné à éviter les divisions par zéro. L'image adversariale finale est ensuite projetée dans l'espace valide des pixels via l'opération $x_{adv} = \text{clamp}(x + \delta, 0, 1)$. Dans les premières expériences, la perte totale était simplement calculée comme la moyenne des pertes sur les images naturelles et sur les images perturbées.

4.1.2 Transition vers l'algorithme TRADES et la divergence KL

Afin de dépasser les limites de cette approche classique, la fonction de coût a été reformulée en s'inspirant de l'algorithme TRADES (Zhang et al., 2019) [10]. Le principe fondamental de TRADES repose sur l'idée que la robustesse ne se résume pas à prédire correctement la classe d'une image attaquée : elle exige avant tout que le modèle soit localement stable, c'est-à-dire que ses prédictions ne varient pas significativement sous de faibles perturbations dans un voisinage L_2 .

Pour opérationnaliser ce principe, la moyenne basique a été remplacée par une fonction de perte intégrant la divergence de Kullback-Leibler (D_{KL}), un outil mathématique permettant de quantifier l'écart entre deux distributions de probabilités. Pour K classes, elle se définit comme :

$$D_{KL}(P(x) \| P(x_{adv})) = \sum_{i=1}^K P(x)_i \cdot \log\left(\frac{P(x)_i}{P(x_{adv})_i}\right) \quad (6)$$

Ce terme est intégré dans la perte totale $\mathcal{L}_{total}$ de la façon suivante :

$$\mathcal{L}_{total} = \mathcal{L}_{CE}(f(x), y) + \beta \cdot D_{KL}(P(x) \| P(x_{adv})) \quad (7)$$

Le paramètre β joue ici un rôle central : il gouverne l'équilibre entre précision naturelle et robustesse adversariale. L'avantage de la divergence KL sur une erreur classique réside dans le fait qu'elle pénalise tout écart dans la distribution de confiance du modèle sur l'ensemble des classes, et non uniquement sur la prédiction finale. Le réseau est ainsi contraint de produire des réponses quasi-identiques pour l'image propre x et son homologue perturbée x_{adv} , ce qui se traduit par un lissage des décisions dans le voisinage de chaque point et une résistance accrue aux attaques.

4.1.3 Régularisation par Lissage des Étiquettes (Label Smoothing)

En complément de l'entraînement adversarial, le Label Smoothing a été introduit afin de prévenir le phénomène de sur-confiance (*overconfidence*) du modèle. Plutôt que d'assigner des cibles dures (1 pour la classe correcte, 0 pour les autres), les objectifs sont adoucis selon la formulation de Szegedy et al. (2016) [11]. La cible y associée à chaque image est redéfinie comme suit :

$$\begin{cases} (1 - \alpha) & \text{si } i = \text{classe cible} \\ \alpha / (K - 1) & \text{sinon} \end{cases} \quad (8)$$

avec $\alpha = 0,1$ comme facteur de lissage et $K = 47$ classes. Cette technique présente deux effets bénéfiques majeurs du point de vue de la robustesse adversariale :

- Contrôle des logits : La perte Cross-Entropy classique pousse la confiance du modèle vers l'infini pour atteindre une perte nulle. En imposant une cible de 0,9 au lieu de 1, la norme des paramètres internes est contenue, ce qui génère des gradients $\nabla_x \mathcal{L}$ moins extrêmes et réduit mécaniquement l'efficacité des attaques basées sur le gradient, telle que la perturbation L_2 utilisée ici.

- Regroupement de l'espace des caractéristiques : Conformément aux observations de Müller et al. (2019) [12], cette régularisation favorise une meilleure cohésion des représentations au sein de chaque classe, créant des marges de décision plus larges. L'adversaire doit alors mobiliser un budget de perturbation δ significativement plus important pour franchir ces frontières et induire une erreur de classification.

En régularisant la dynamique d'apprentissage, le Label Smoothing constitue un préalable favorable à l'action de la divergence KL de TRADES, en évitant que celle-ci n'ait à compenser des gradients numériquement instables.

4.1.4 Progression du Budget de Perturbation (Scheduling)

Un planificateur (*scheduler*) a également été mis en place pour faire varier progressivement l'amplitude de l'attaque ϵ au cours de l'entraînement. Cette approche s'inspire des travaux de Cai et al. (2018) [13] dans leur article "Curriculum Adversarial Training", qui démontrent qu'une augmentation graduelle de la difficulté adversariale stabilise l'apprentissage et prévient l'effondrement des performances face à des perturbations trop fortes en début d'entraînement.

Deux stratégies de progression ont été comparées :

- Croissance linéaire : L'amplitude de l'attaque croît de manière uniforme. Cette approche, bien que simple à mettre en œuvre, s'avère trop rigide en fin d'entraînement.
- Croissance sinusoïdale : Une dynamique non-linéaire est appliquée via la fonction $f(t) = \sin(\frac{\pi}{2} \cdot \frac{t}{T})$. Cette formulation accélère la montée en difficulté en début d'entraînement afin d'échapper rapidement aux minima locaux, puis ralentit la progression à l'approche du budget maximal ϵ_{max} .

Les expériences menées indiquent que la progression sinusoïdale produit de meilleurs résultats. Il convient néanmoins de souligner que d'autres stratégies de scheduling — telles que des croissances exponentielles ou par paliers — n'ont pas encore été explorées et pourraient offrir des compromis intéressants entre vitesse de convergence et robustesse finale.

4.1.5 Évaluation croisée et optimisation des hyperparamètres

Un protocole d'évaluation double a été mis en place pour piloter l'optimisation des hyperparamètres. À chaque évaluation, deux métriques sont mesurées : la précision sur les données naturelles et la précision sur les données adversariales. Ce suivi conjoint a permis d'identifier les configurations maximisant la robustesse sans dégrader significativement les performances sur les données saines.

Deux hyperparamètres spécifiques à la méthode proposée ont nécessité une calibration rigoureuse : le paramètre β de TRADES et le facteur de lissage α . Une valeur de β trop élevée favorise la robustesse au détriment de la précision naturelle, tandis qu'un α mal calibré nuit à la capacité du modèle à capter les détails discriminants des images.

Les résultats obtenus pour différentes configurations sont présentés dans le tableau suivant :

TABLE 4 – Analyse de la robustesse selon β et le Label Smoothing

Hyperparamètres		Acc. Nat.	Précision Adversariale vs ϵ (L_2)				
β	L. Smoothing (α)	Saine	0.1	0.2	0.3	0.4	0.5
0.5	0.9	90.4%	89.0%	87.6%	86.3%	84.9%	83.6%
0.5	0.8	90.3%	89.0%	87.7%	86.5%	85.2%	83.8%
1.0	0.9	90.1%	89.0%	87.8%	86.8%	85.6%	84.4%
1.0	0.8	90.1%	89.0%	88.0%	86.8%	85.7%	84.5%
3.0	0.9	89.7%	88.0%	87.0%	85.9%	85.0%	84.0%
3.0	0.8	89.6%	88.1%	87.1%	86.2%	85.2%	84.3%

Conformément aux prédictions théoriques de TRADES, une valeur de β plus faible favorise la précision naturelle, avec un maximum de 90,4% pour $\beta = 0,5$. Un résultat notable est que la configuration $\beta = 1,0$ surpasse $\beta = 3,0$ en termes de robustesse finale. Ce phénomène s'explique probablement par le fait qu'à $\beta = 3,0$, la dégradation de la précision naturelle (89,7%) n'est pas suffisamment compensée par le gain en stabilité locale, ce qui abaisse mécaniquement le plafond de performance global face aux attaques.

Par ailleurs, les résultats ont l'air de montrer qu'un facteur de lissage $\alpha = 0,8$ améliore systématiquement la résistance du modèle face aux perturbations les plus importantes ($\epsilon = 0,5$), au prix d'une légère diminution de la précision sur les données naturelles. Comme nous voulons un compromis entre

robustesse et performance nous avons choisis de prendre $\beta = 1$ car ils obtient une performance saine proche de 0.5 tout en ayant une belle améliorations face au attaques plus forte, pour α nous avons choisi de le mettre à 0.8 car les résultats sains semblent les même quand $\beta = 1$ mais les résultats sont meilleurs pour les images attaquées .

4.1.6 Résultats finaux de robustesse

Notre modèle final robuste améliore largement les résultats comparés au classifieur initial. D'abord, sur les données naturelles, le modèle robuste atteint une précision de 90,1 %, légèrement supérieure aux 89,5 % du modèle initial, ce qui confirme que les techniques de régularisation employées n'ont pas nui à la capacité de généralisation du réseau. L'apport le plus significatif se manifeste face aux attaques adversariales. Comme l'illustre la figure 7, face à l'attaque PGD+, le modèle robuste maintient une précision de 80 % à $\varepsilon = 0,7$, là où le modèle initial s'effondre à environ 50 %. Sur l'ensemble des valeurs de ε testées, le gain moyen de robustesse face à PGD+ s'établit à +10 à +15 points de pourcentage. Face à l'attaque CW (figure 9), les deux modèles maintiennent une précision voisine de 90 %, ce qui s'explique par les contraintes inhérentes à cette attaque détaillées en section 3.2.2. Malgré cela, on remarque que la version robuste a parfois de moins bons résultats que la version initiale, mais qu'en moyenne elle subit moins les attaques ; ces moins bons résultats s'expliquent peut-être par un simple hasard. Ces résultats valident l'efficacité de la combinaison TRADES + Label Smoothing + scheduling sinusoïdal : le modèle robuste offre une résistance nettement accrue aux perturbations adversariales tout en conservant, voire en améliorant légèrement, ses performances sur données saines.

5 Conclusion

Ce projet nous a permis de nous plonger au coeur du machine learning et des réseaux de neurones. Nous avons également pu développer diverses compétences et connaissances utiles pour la suite de notre parcours.

D'abord, nous n'avions jamais été confronté aux réseaux neuronaux, et l'entraînement de ceux-ci. Une première phase a donc d'abord été de nous familiariser avec ces concepts (tels que la rétro-propagation, descente de gradient, fonctions de coût, etc.), et créer une première version, utilisant un Perceptron Multicouches (MLP). Ensuite, pour pouvoir exploiter la structure spatiale des images, nous avons évolué vers une approche CNN, plus performante et certainement plus robuste pour la seconde partie du projet.

En seconde partie, nous nous sommes penchés sur les attaques adversariales, ce qui nous a permis de constater la vulnérabilité importantes de notre modèle, sensible à de minuscules changements invisibles à l'œil nu. Nous avons développé deux approches, PGD+ et C&W, et avons essayé de formuler précisément leurs objectifs respectifs d'optimisation.

Face à cette menace, nous avons finalement essayé d'être moins perméables à de telles attaques, en modifiant la structure et les hyperparamètres de notre modèle d'abord, et le dataset. Nous avons d'abord modifié l'objectif de notre entraînement, en changeant notre fonction de coût pour également refléter la résistance à de faibles perturbations (TRADES). De plus, nous avons également commencé à entraîner notre modèle sur des images perturbées par nos propres attaques.

Au delà des performances de notre modèle, ce projet nous a surtout permis de découvrir un nouveau champs d'ingénierie, et d'affiner notre débrouillardise, en nous obligeant à fouiller à travers la littérature scientifique sur le sujet et les nombreuses sources disponibles. Nous avons ainsi appris à gérer le compromis entre temps d'exécution et convergence en ajustant le batch size, à modifier dynamiquement le taux d'apprentissage, de même que différentes stratégies de recherche d'hyperparamètres.

6 Utilisation de l'IA

Au cours de ce projet, nous avons eu recours à de l'IA générative pour différentes utilisations, y compris les suivantes :

- Compréhension : Questions relatives à la compréhension de cette matière incroyablement vaste et nouvelle pour nous, qu'est le machine learning
- Sources : Demandes de fourniture de sources adéquates pour nos différentes pistes, ainsi qu'utilisation d'un NotebookLM afin de regrouper, organiser et résumer nos différentes sources
- Code : Suggestions d'améliorations de code (optimisation, clarté, structure) et corrections d'erreurs
- Rapport : Feedback rédactionnel, comme par exemple une vérification finale sur les incohérences et fautes d'orthographe de ce rapport

Cependant, nous nous engageons à ne pas avoir utilisé l'IA comme un remplaçant, mais plutôt comme un assistant. Nous avons recherché les sources des propositions d'améliorations formulées, et ensuite implémenté celles-ci, pas simplement accepté les modifications sans les comprendre.

Références

- [1] Grant SANDERSON. *Neural Networks*. Playlist YouTube: 3Blue1Brown. 2017. URL : https://www.youtube.com/playlist?list=PLZHQB0WTQDNU6R1_67000Dx_ZCJB-3pi (visité le 02/03/2026).
- [2] *CrossEntropyLoss — PyTorch Documentation*. <https://pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html>. Accessed: 2026-03-03.
- [3] Kevin JAMIESON et Ameet TALWALKAR. "Non-stochastic Best Arm Identification and Hyperparameter Optimization". In : *Proceedings of the 19th International Conference on Artificial Intelligence and Statistics*. Sous la dir. d'Arthur GRETTON et Christian C. ROBERT. T. 51. Proceedings of Machine Learning Research. Cadiz, Spain : PMLR, sept. 2016, p. 240-248. URL : <https://proceedings.mlr.press/v51/jamieson16.html>.
- [4] Lisha LI et al. "Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization". In : *Journal of Machine Learning Research* 18.185 (2018), p. 1-52. URL : <http://jmlr.org/papers/v18/li16-558.html>.
- [5] Stefan FALKNER, Aaron KLEIN et Frank HUTTER. "BOHB: Robust and Efficient Hyperparameter Optimization at Scale". In : *Proceedings of the 35th International Conference on Machine Learning*. Sous la dir. de Jennifer DY et Andreas KRAUSE. T. 80. Proceedings of Machine Learning Research. PMLR, oct. 2018, p. 1437-1446. URL : <https://proceedings.mlr.press/v80/falkner18a.html>.
- [6] Datenzauber AI. *Do Batch Sizes Actually Need To Be Powers of 2?* Accessed: 2026-03-23. 2024. URL : <https://wandb.ai/datenzauberai/Batch-Size-Testing/reports/Do-Batch-Sizes-Actually-Need-to-be-Powers-of-2---VmlldzoyMDkwNDQx>.
- [7] Pranav JEEVAN et Amit SETHI. "WaveMix: A Resource-efficient Neural Network for Image Analysis". In : *arXiv preprint arXiv:2205.14375* (2022). Updated 2024. URL : <https://arxiv.org/pdf/2205.14375>.
- [8] Francesco CROCE et Matthias HEIN. "Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks". In : *International Conference on Machine Learning (ICML)*. Référence pour l'optimisation des paramètres d'attaque (restarts et itérations) en fonction du budget de perturbation. PMLR. 2020, p. 2206-2216.
- [9] Nicholas CARLINI et David WAGNER. *Towards Evaluating the Robustness of Neural Networks*. 2017. arXiv : 1608.04644 [cs.CR]. URL : <https://arxiv.org/abs/1608.04644>.
- [10] Hongyang ZHANG et al. "Theoretically Principled Trade-off between Robustness and Accuracy". In : *International Conference on Machine Learning*. PMLR. 2019, p. 7472-7482.
- [11] Christian SZEGEDY et al. "Rethinking the Inception Architecture for Computer Vision". In : *Proceedings of the IEEE conference on computer vision and pattern recognition (CVPR)*. 2016, p. 2818-2826.
- [12] Rafael MÜLLER, Simon KORNBLITH et Geoffrey E HINTON. "When does label smoothing help?" In : *Advances in neural information processing systems (NeurIPS)* 32 (2019).
- [13] Qi-Zhi CAI et al. "Curriculum Adversarial Training." In : *IJCAI*. T. 18. 2018, p. 3740-3747.